

Ray tracing w C++

Jakub Soboń

Czerwiec 2026

1 Cel projektu

Celem projektu było napisanie prostego silnika renderującego wykorzystującego śledzenie promieni wstecznych. Program generuje obraz sceny trójwymiarowej, wypuszczając z kamery promień dla każdego piksela i sprawdzając jego przecięcia z obiektami. Na podstawie najbliższego trafienia obliczany jest kolor piksela.

W implementacji uwzględniamy odbicia zwierciadlane, pomimo niedokładności takiego modelu względem prawdziwego życia ale wystarcza on do pokazania najważniejszej cechy metody czyli tego promień może odbijać się rekurencyjnie od wielu powierzchni, dzięki czemu obiekty "widzą" siebie nawzajem w odbiciach.

2 Model sceny

Scena składa się z kul oraz tła w postaci pionowego gradientu jak i dużej kuli na dole będącej refleksyjną podłogą. Kamera znajduje się w początku układu współrzędnych i patrzy wzdłuż dodatniej osi Z . Płaszczyzna rzutowania znajduje się przed kamerą, a jej rozmiar określa pole widzenia. Dla każdego punktu tej płaszczyzny tworzony jest promień o początku w kamerze i kierunku prowadzącym przez dany piksel tak jak w opisie.

Promień zapisany jest w postaci:

$$\mathbf{P}(t) = \mathbf{A} + t\mathbf{B}, \quad t \geq 0, \quad (1)$$

gdzie $\mathbf{A}$ oznacza początek promienia, a $\mathbf{B}$ jego znormalizowany kierunek. Kula jest

opisana środkiem $\mathbf{C}$ i promieniem r :

$$|\mathbf{P}(t) - \mathbf{C}|^2 = r^2. \quad (2)$$

Po podstawieniu równania promienia otrzymuje się równanie kwadratowe. Jeżeli delta jest ujemna, promień nie trafia w kulę. W przeciwnym razie wybierany jest najmniejszy dodatni pierwiastek, czyli najbliższe widoczne przecięcie.

3 Odbicia

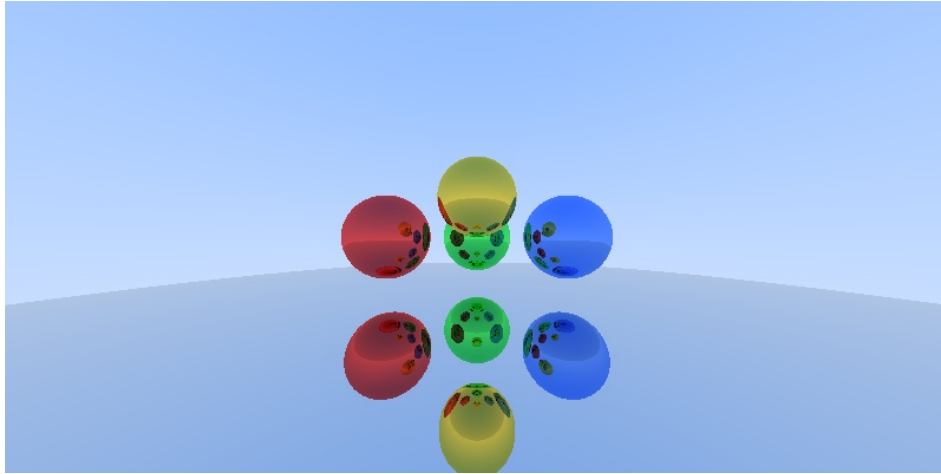
Kluczowym elementem programu jest materiał zwierciadlany. Dla promienia padającego $\mathbf{I}$ i powierzchni normalnej $\mathbf{N}$ kierunek odbicia wyznaczany jest wzorem:

$$\mathbf{R} = \mathbf{I} - 2(\mathbf{I} \cdot \mathbf{N})\mathbf{N}. \quad (3)$$

Następnie silnik śledzi nowy promień w kierunku $\mathbf{R}$. Kolor obiektu działa jak filtr: czerwone lustro przepuszcza głównie składową czerwoną odbitego światła, zielone zieloną itd.

W kodzie odpowiadają za to głównie klasy `Ray`, `Sphere`, `Scene`, `MirrorMaterial` oraz `Renderer`. Klasa `Scene` wyszukuje najbliższe przecięcie, a `Renderer` steruje rekurencją. Limit głębokości odbić ustawiono na 10, aby uniknąć nieskończonego odbijania promieni między powierzchniami.

Zastosowano również małe przesunięcie punktu startowego promienia odbitego wzdłuż normalnej. Ma to za zadanie uniknięcia ponownego trafienia promienia w tę samą powierzchnię z powodu błędów zaokrągleń.



Rysunek 1: Wynik po działaniu programu, widoczne są odbicia kul oraz gradientowego tła na kulach i podłodze.

4 Wynik

Scena testowa zawiera cztery mniejsze kule o różnych barwach oraz lustrzaną podłogę. Wynik zapisano najpierw jako plik PPM, a następnie przekonwertowano do formatu JPEG w celu wygodnego umieszczenia w raporcie. Na renderze widać poprawne przecięcia promieni z kulami, ostre krawędzie obiektów oraz rekurencyjne odbicia. Szczególnie dobrze widać odbicia bocznych kul na powierzchni kuli przedniej i na podłodze. Brak widocznych plam przy punktach styku potwierdza, że przesunięcie startu promienia odbitego działa poprawnie.

czyli programem CMake oraz w sprawdzeniu gramatyki i interpunkcji raportu CMake został użyty z uwagi na fakt że na moim prywatnym komputerze miałem za dużo Pathów w złej kolejności.

5 Wnioski

Program działa zgodnie z wymaganiami. Implementuje silnik śledzenia promieni, obsługuje przecięcia ze sferami, odbicia zwierciadlane, jak i zapis obrazu. Można dodać matowe powierzchnie wtedy odbicia muszą być w dowolnym kierunku spełniając przy tym prawo cosinusów Lamberta.

6 Uwagi nieformalne

Sztuczna inteligencja została użyta do pomocy z generatorem systemu budowania